

Why a 24-layer stream-bottleneck model does not train

Research journal

Started 2026-09-30 (box lisbon, 8×RTX 5090).

Contents

1	The question	1
2	Entry 1 — the gains of one bottleneck	2
3	Entry 2 — consequences at depth	3
4	Entry 3 — removing the selection gain (at initialization)	3
5	Entry 4 — round 1: the post-norm variants still do not train	4
6	Entry 5 — forensics of the failed run at step 2000	4
7	Entry 6 — the second mechanism: a renormalized carry	5
8	Entry 7 — what a stream bottleneck must do: the skip’s job	7
9	Entry 8 — round 2: controls, depth, and the value shift in training	8
10	Note on how the two fixes were arrived at	8
11	Implementation notes	9

1 The question

Every block of the models studied here ends in a stream bottleneck (`placement=residual_out`): the residual stream is replaced by

$$B(h) = g_D W_D P_{S(h)} W_E h, \quad S(h) = \text{TopK}_K(|W_E h|), \quad (1)$$

where $W_E \in \mathbb{R}^{N \times d}$ is the encoder, $W_D \in \mathbb{R}^{d \times N}$ the decoder, P_S the coordinate projection onto the K kept features and g_D a fixed scalar ($\sqrt{d/K}$ under `bottleneck_decoder_scale=backward_preserving`, “bp”). Nothing routes around B : the stream entering block $\ell + 1$ is $x_{\ell+1} = \mathcal{N}(B(h_\ell))$ with $h_\ell = x_\ell + \Delta_\ell$, Δ_ℓ the attention and MLP contributions of block ℓ , and $\mathcal{N}$ either the identity or an RMSNorm (`post_norm`). Throughout: $d = 1024$, $N = 4096$, $K = 512$ unless stated, magnitude–direction decoupling (MD), TinyStories, lr $3 \cdot 10^{-4}$, cosine over 20k steps, batch 64×512 .

Observed before this study:

run	layers	outcome
al_8L_hard_k512_nopnorm_bp_std	8	trains: val CE 3.46 (500), 2.34 (2k), 1.418 (20k)
al_500m_hard_k512_pnorm_bp_std	24	val CE 5.965 (500) $\rightarrow$ 5.949 (10.5k), flat
po_500m_hard_k512_pnorm_bp_std_shared	24	same plateau (5.954 at 2.5k)
al_500m_hard_k32_pnorm_md	24	same plateau (5.947 at 4k)
al_500m_hard_k512_nopnorm_bp_std	24	CE = 10.8249 = $\ln 50257$ from step ~ 380 , zero gradient
al_500m_hard_k512_nopnorm_bp_ortho	24	the same, from step ~ 560
al_500m_hard_k512_nopnorm_bp_std_246e4	246e4	the same

The plateau is the unigram entropy. The token-frequency entropy of `train.bin` is 5.9665 nats (and the cross-entropy of `val.bin` under the train unigram is 5.9665). The post-norm 24-layer runs therefore learn the token frequencies within ~ 150 steps and nothing that depends on the input afterwards. The no-post-norm runs end at exactly $\ln V$: uniform logits.

2 Entry 1 — the gains of one bottleneck

Two exact properties of (1). Abs-TopK selection does not change under a positive rescaling of h , so $B(\alpha h) = \alpha B(h)$; and B is piecewise linear, $B(h) = J(h)h$ with $J(h) = g_D W_D P_{S(h)} W_E$, which is also the Jacobian autograd uses (the mask has zero derivative almost everywhere).

Two gains. Per token, the forward (signal) gain and the backward gain of a gradient g arriving at the output are

$$G_f = \frac{\|B(h)\|^2}{\|h\|^2}, \quad G_b = \frac{\|J(h)^\top g\|^2}{\|g\|^2}. \quad (2)$$

At initialization the entries of W_E, W_D are i.i.d. $\mathcal{N}(0, 1/d)$ (projected to the MD sphere), so the pre-activations $z = W_E h$ are Gaussian with variance $\sigma^2 = \|h\|^2/d$, and the decoder columns are orthonormal up to $O(d^{-1/2})$. Then $\|B(h)\|^2 \approx g_D^2 \sum_{i \in S} z_i^2$, while a gradient that the selection did not see (isotropic, independent of S) puts $\|g\|^2/d$ on each kept coordinate:

$$G_f \approx g_D^2 \frac{K}{d} s^2(\rho), \quad G_b \approx g_D^2 \frac{K}{d}, \quad s^2(\rho) = \mathbb{E}[Z^2 \mid |Z| > t_\rho] = 1 + \frac{2t_\rho \varphi(t_\rho)}{\rho}, \quad (3)$$

with $\rho = K/N$, $t_\rho = \Phi^{-1}(1 - \rho/2)$ and φ the standard normal density. s^2 is the *selection gain*: the kept coefficients are the largest ones, so their mean square exceeds that of a typical coefficient. $s^2 = 4.019$ at $\rho = 1/8$ and 8.898 at $\rho = 1/128$ ($K = 32$).

Check against the existing K sweep (docs/figures/diag500m/module_gain, 24 layers at init, geometric mean over layers):

K	$s^2(K/N)$	G_f/G_b (post-norm, $g_D = 1$)	G_f (bp, ortho)	G_b (bp, ortho)
32	8.898	9.159	8.869	2.156
128	6.407	6.474	6.401	1.510
512	4.019	4.033	4.018	1.006
1024	2.894	2.926	2.896	0.999
2048	1.857	1.871	1.858	1.000

With a post-norm the ratio G_f/G_b equals $s^2(K/N)$ to 0.5–3% at every K . Without one (right columns) G_f still equals s^2 to 0.3%; G_b exceeds 1 at small K because there the gradient arriving from the next bottleneck is no longer isotropic (it is concentrated on the direction of the signal, which receives the forward gain).

Invariance. Multiplying W_E , W_D or g_D by scalars multiplies G_f and G_b by the same factor; an RMSNorm after B divides both by G_f ; a tight-frame (orthogonal) initialization leaves the marginal of z Gaussian; sharing W_E, W_D across layers changes neither. The ratio $G_f/G_b = s^2$ is therefore unchanged by every intervention tried before this study (post-norm, bp scale, MD gain spread α , orthogonal init, shared projections). Measured at init with `analysis/depth_probe.py`: the per-layer factor of the gradient profile is $\times 2.095$ (baseline), $\times 2.096$ (shared), $\times 2.102$ (orthogonal).

3 Entry 2 — consequences at depth

Without a norm the stream energy grows as $\prod_\ell G_f \approx s^{2L}$ (with bp). $4.02^{24} \approx 3 \cdot 10^{14}$: measured stream RMS at the last layer at init $1.6 \cdot 10^7$ (predicted $4.02^{12} = 1.7 \cdot 10^7$). The final RMSNorm divides the loss gradient by that RMS, so every parameter gradient is $10^{-11} - 4 \cdot 10^{-8}$ (fc1 weight, layer 23 and layer 0), at or below Adam's $\epsilon = 10^{-8}$. Once training grows any gain, x^2 exceeds the float32 range inside the RMSNorm ($|x| > 1.8 \cdot 10^{19}$), its output becomes zero and so do the logits: $CE = \ln V$ and zero gradient, as in the three no-post-norm runs (their logged boundary gaps reach $10^{18} - 10^{24}$). At 8 layers the growth is $4^{8/2} = 256$ in RMS (measured 238) and training works.

With a post-norm the forward is normalized, and the backward through each post-norm is divided by the same $\sqrt{G_f}$: per layer the gradient energy is multiplied by $G_b/G_f = s^{-2}$. The gradient reaching block ℓ is $s^{-(L-\ell)}$ times the last block's. Measured at init (fc1 weight gradient RMS): $6.7 \cdot 10^{-12}$ at layer 0 against $1.6 \cdot 10^{-4}$ at layer 23, a ratio $2.4 \cdot 10^7$ ($\times 2.095$ per layer; $s = 2.005$). At 8 layers the ratio is 247.

4 Entry 3 — removing the selection gain (at initialization)

Two ways to make the forward stop carrying the selection bias, both now options of `ActivationBottleneckConf`

(a) **Debiased values, value_shift.** Keep the support, shrink the kept magnitudes,

$$y_i = \text{sign}(z_i) \max(|z_i| - \delta, 0), \quad i \in S. \quad (4)$$

The tangent Jacobian on the support is unchanged, so G_b is unchanged, while G_f drops. **energy** solves δ per token from $\text{mean}_S y^2 = \text{mean}_{\text{all}} z^2$, which gives $G_f = G_b$ exactly for a tight-frame encoder; **fixed** uses $\delta = \lambda^* \text{RMS}(z)$ with λ^* the root of $\mathbb{E}[(|Z| - \lambda)^2 \mid |Z| > t_\rho] = 1$ ($\lambda^* = 1.044$ at $\rho = 1/8$).

(b) **Code-space residual, code_residual.** Carry the K -sparse code instead of re-encoding the decoded stream:

$$c_0 = \text{TopK}(W_E x_{\text{emb}}), \quad x_\ell = g_D W_D c_\ell, \quad c_{\ell+1} = \text{TopK}(c_\ell + \alpha W_E \Delta_\ell(x_\ell)), \quad (5)$$

one shared dictionary. `TopK` is the Euclidean projection onto the set of K -sparse vectors, so a block with $\Delta_\ell = 0$ is the identity on the code, and $\partial c_{\ell+1} / \partial c_\ell = P_{S_{\ell+1}}(I + \dots)$: the residual connection of a pre-norm transformer, moved into code space.

Per-layer factor of the init gradient profile (fc1 weight gradient RMS, $(g_{23}/g_0)^{1/23}$); 1 is flat:

variant	per-layer factor	last-layer stream RMS
baseline, post-norm	2.095	1.0
baseline, no post-norm	0.697	$1.6 \cdot 10^7$
value shift fixed $\lambda = 0.7$, post-norm	1.387	1.0
value shift fixed $\lambda^* = 1.044$, post-norm	1.044	1.0
value shift fixed $\lambda = 1.3$, post-norm	0.798	1.0
value shift energy , post-norm	1.044	1.0
value shift energy , no post-norm	0.939	6.4
code residual $\alpha = 1 / 0.3 / 0.1$	1.008 / 0.995 / 0.997	8.7 / 3.0 / 2.1
rb lapsum surrogate $T = 0.3 / 1 / 3$, post-norm	0.709 / 1.070 / 1.445	1.0
lapsum surrogate $T = 1$, post-norm	1.567	1.0

With the shift, the bottleneck’s measured G_f and G_b are both 1.00 ± 0.01 at every layer. The surrogate rows show the other side of the same balance: a sharp surrogate ($T = 0.3$) adds boundary-exchange gradient and the backward explodes toward the input, a wide one vanishes; $T \approx 1$ is close to critical for the backward only (its forward is the plain hard TopK).

5 Entry 4 — round 1: the post-norm variants still do not train

Eight 24-layer runs, the baseline’s config otherwise, screened to 2000 steps (`--stop-step`; the schedule stays defined over 20k steps). Train CE:

run	100	200	300	500	1000
baseline (al_500m_hard_k512_pnorm_bp_std)	6.43	5.98	5.97	5.98	5.99
value shift energy, post-norm	6.40	5.99	5.97	5.98	
value shift fixed, post-norm	6.40	5.98	5.97	5.99	
value shift energy, post-norm, shared	6.42	5.98	5.98	5.99	
value shift energy, post-norm, $K = 32$	7.11	5.98	5.97		
rb lapsum $T = 1$, post-norm	6.41	5.98	5.97	5.98	
value shift energy, <i>no</i> post-norm	6.53	5.41	4.85	4.25 [†]	
code residual $\alpha = 1$	5.25	3.79	3.28	2.79	2.19
code residual $\alpha = 0.3$	5.13	3.75	3.29	2.84	2.21
8 layers, no post-norm (reference)	6.72	4.53	4.06	3.53	2.89

[†]val CE at step 500.

Measured: every run with a post-norm follows the baseline to the second decimal, including those whose initial gradient profile is flat (value shift) or near-critical (rb lapsum). Both runs without a post-norm leave the plateau from the start; the code residual is ahead of the 8-layer reference at every step. **Interpretation:** removing the selection gain is necessary (the no-post-norm baseline overflows) but not sufficient; the post-norm itself stops learning at this depth.

6 Entry 5 — forensics of the failed run at step 2000

analysis/depth_probe.py --ckpt ... --init-reference on al_500m_hard_k512_pnorm_bp_std/ckpt_step2000.pt and, for contrast, al_8L_hard_k512_nopnorm_bp_std/ckpt_step2000.pt. $\Delta W/W_0 = \|W - W_0\|/\|W_0\|$ against the run’s own step-0 weights (rebuilt with the same seed; md_init_ draws on the GPU, as in train()).

	layers 0–12	layer 18	layer 21	layer 23
$\Delta W/W_0$, fc1 (24 layers, post-norm)	$< 5 \cdot 10^{-4}$	0.003	0.022	0.204
fc1 gradient RMS	10^{-20} – 10^{-16}	$5 \cdot 10^{-15}$	$2 \cdot 10^{-12}$	$5 \cdot 10^{-8}$
cosine between positions	0.07–0.21	0.23	0.992	1.000
bottleneck G_f	3.97–4.05	4.64	81	2680
$\Delta W/W_0$, fc1 (8 layers, no post-norm)	0.30–0.54 at every layer; cosine between positions ≤ 0.17			

Measured: after 2000 steps the first 13 blocks of the 24-layer run have not moved (Adam with $\epsilon = 10^{-8}$ and gradients of 10^{-20} – 10^{-16}); the per-layer gradient factor has grown from 2.1 at init to 3.5. In the last three blocks the bottleneck’s forward gain rises to 81–2680 (their outputs carry 56–18861 times the energy of the unit carry, Entry 6), and the streams of all positions are the same vector (cosine 1.000): the output cannot depend on the input, which is the unigram plateau.

7 Entry 6 — the second mechanism: a renormalized carry

Common-mode fraction. For the streams v_t of the positions of a sequence let $f(v) = \|\bar{v}\|^2 / \overline{\|v_t\|^2}$, $\bar{v}$ the mean over positions ($t \geq 16$). For $v_t = c + u_t$ with independent u_t this is $\|c\|^2 / (\|c\|^2 + \|u\|^2)$, the expected cosine between two positions, and it is what `depth_probe.py` records as `f_stream` (and, sampled, as `tok_cos`). Per block it also records $r = \|\Delta\|^2 / \|x\|^2$ and $r_c = \|\bar{\Delta}\|^2 / \|x\|^2$, the energy of the block’s contribution and of its token-independent part relative to the carry.

Model. Adding Δ gives $f(h) \approx (f(x) + r_c)/(1 + r)$. The bottleneck maps a small common fraction by the slope of its correlation map; for Gaussian codes and $\phi(z) = z \mathbf{1}[|z| > t]$ the Hermite expansion gives

$$\kappa = C'_B(0) = \frac{\mathbb{E}[Z\phi(Z)]^2}{\mathbb{E}[\phi(Z)^2]} = \mathbb{E}[Z^2 \mathbf{1}(|Z| > t_\rho)] = \rho s^2(\rho), \quad (6)$$

the Gaussian energy fraction that survives the selection (0.50 at $\rho = 1/8$, 0.070 at $K = 32$, 1 for $K = N$). An RMSNorm leaves angles, and so f , unchanged, but it resets the carry to unit RMS, so r and r_c stay $O(1)$ at every depth and

$$f_{\ell+1} = \kappa \frac{f_\ell + r_c}{1 + r} \implies f^* = \frac{\kappa r_c}{1 + r - \kappa}, \quad (7)$$

approached geometrically. Without a norm the carry accumulates energy, r_ℓ falls with depth, and there is no fixed point pulling f toward 1.

Measured at initialization (24 layers, `init3/`; r, r_c averaged over blocks 6–23, f over blocks 12–23):

	$\kappa = \rho s^2$	κ measured	r_c	r	f^* (7)	f measured
post-norm, $K = 32$	0.070	0.092	0.37	1.74	0.010	0.013
post-norm, $K = 512$	0.502	0.544	0.88	1.91	0.183	0.197
post-norm, $K = 2048$	0.929	0.955	2.39	2.56	0.843	0.883
post-norm, $K = N$	1	1.000	2.62	2.66	0.985	0.989
no norm, $K = N$	1	1.002	r : 0.17 $\rightarrow$ 0.05		–	0.56
dense, no bottleneck	–	1.000	r : 0.16 $\rightarrow$ 0.05		–	0.59
value shift, no norm	0.502	0.520	r : 0.15 $\rightarrow$ 0.04		–	0.009
code residual	–	0.883	r : 0.06 $\rightarrow$ 0.02		–	0.06

Measured: with a post-norm, (7) with the *theoretical* κ predicts the common-mode fraction to within 0.003–0.04 from $K = 32$ to $K = N$. With $K = N$ (a fixed random linear carry, no selection) the stream is token-independent already at initialization ($f = 0.99$); the same carry without the

post-norm stays at 0.56. Most of r_c comes from the MLP (GELU has a nonzero mean): the attention’s share is 0.2 of 0.9 at $K = 512$. **Interpretation:** the selection’s decorrelation ($\kappa < 1$) is what keeps the $K = 512$ post-norm stack at $f \approx 0.2$ at initialization. Training makes the top blocks emit a large token-independent output (the unigram): their r_c grows (to 56–18861 times the carry energy in the failed run, table below), the selection becomes the same for every position ($\kappa \rightarrow 1$), and $f \rightarrow 1$ (cosine 1.000 at step 2000, Entry 5). The large G_f of those blocks also divides the gradient reaching everything below them, which is why the per-layer gradient factor grew from 2.1 to 3.5.

The same quantities in the failed run (a1_500m_hard_k512_pnorm_bp_std, $\kappa_\ell = f_{\ell+1}/f(h_\ell)$ measured):

block	0–17	18	19	20	21	22	23
f (step 2000)	0.07–0.23	0.23	0.36	0.80	0.992	1.000	1.000
κ	0.54–0.56	0.78	1.07	1.006	1.000	1.000	–
r_c	0.47–0.90	1.19	3.04	15.3	56.4	528	18861
r_c/r	0.27–0.47	0.55	0.82	0.995	1.000	1.000	1.000
selection energy s^2	4.02	4.06	4.53	6.24	7.79	8.00	7.98

Blocks 0–17 are still at their initialization values (they did not move). The blocks that trained emit an almost purely token-independent output ($r_c/r \rightarrow 1$); their inputs are then exactly K -sparse in the encoder ($s^2 \rightarrow N/K = 8$: all of the energy on the kept features), the support is the same for every position ($\kappa \rightarrow 1$), and $f \rightarrow 1$. The recursion (7) reproduces it block by block (block 20: $(0.80 + 15.3)/16.4 = 0.98$, measured 0.992). Step 10000 is the same picture with larger top-block energies ($r_c = 5 \cdot 10^5$ at block 23).

The value shift under a post-norm collapses too (li_24L_k512_pnorm_shift_energy, step 2000, val CE 5.959; its initial gradient profile was flat, $\times 1.047$ per block):

block	0	6	12	16	18	20	22	23
f	0.07	0.50	0.67	0.91	0.994	1.000	1.000	1.000
κ	0.56	0.76	0.83	1.00	1.00	1.00	1.00	–
r_c/r	0.28	0.78	0.86	0.98	1.00	1.00	1.00	1.00
bottleneck G_f	1.00	1.07	1.11	1.53	2.23	5.82	244	470
$\Delta W/W_0$, fc1	$5.9 \cdot 10^{-3}$	$6.8 \cdot 10^{-3}$	$6.7 \cdot 10^{-3}$	$8.3 \cdot 10^{-3}$	0.015	0.020	0.063	0.19

Measured: with a balanced bottleneck every block moved in the first steps ($\Delta W/W_0 \approx 6 \cdot 10^{-3}$ at block 0, against $4 \cdot 10^{-8}$ in the baseline), but the token-independent share then spread down the stack — from block 16 on the streams of all positions coincide — and the large forward gain of the collapsed blocks (up to 470) brought the vanishing gradient back ($\times 2.3$ per block at step 2000). **Interpretation:** the renormalized carry is sufficient on its own to stop learning at 24 layers; the selection gain only decides how early it happens.

Training controls ($K = N$, 24 layers): with the post-norm val CE 5.967 (500), 5.956 (1000), on the plateau; without it, *[pending]*.

All runs at step 2000 (depth_probe.py --ckpt on each run’s ckpt_step2000.pt). Per-block gradient factor $(g_{\text{last}}/g_0)^{1/(L-1)}$ of the fc1 weight, relative weight change of block 0 and of the last block, cosine between positions at the last block:

run (24 layers unless marked)	grad. factor	$\Delta W/W_0$, block 0	$\Delta W/W_0$, last	cosine, last
TopK, post-norm (the failed run)	3.48	4.4e-08	0.20	1.000
value shift, post-norm	2.30	5.9e-03	0.19	1.000
$K = N$, post-norm	2.64	2.0e-03	0.16	1.000
value shift, no norm	0.93	3.1e-01	0.54	0.205
code residual	1.02	3.9e-01	0.39	0.120
code residual, $\alpha = 0.3$	1.04	3.6e-01	0.45	0.111
code residual, $K = 32$	1.04	3.8e-01	0.40	0.082
code residual, 8 layers	1.08	3.7e-01	0.41	0.106
TopK, no norm, 8 layers	1.04	3.0e-01	0.54	0.173

Measured: the post-norm decides the outcome. Every run with a post-norm ends with the positions collapsed (cosine 1.000) and a vanishing gradient ($\times 2.3$ – 3.5 per block, whatever its initial profile); every run without one keeps the positions apart (≤ 0.2), a flat gradient ($\times 0.93$ – 1.08), and moves all of its blocks by 0.3–0.5.

8 Entry 7 — what a stream bottleneck must do: the skip’s job

A pre-norm transformer trains at depth because of two properties of its stream: every block is the identity plus a small update (the input–output Jacobian of the stack is close to an isometry), and nothing normalizes the carried stream (only the branch inputs are normalized). A stream bottleneck replaces the skip, so it has to take over both roles. The three requirements, and which variant meets them:

	balanced ($G_f = G_b$)	carry not renormalized	identity when $\Delta = 0$	24 layers tra
TopK, post-norm (the failed runs)	$\times (s^2)$	$\times$	$\times$	$\times$
TopK, no norm	$\times (s^2)$	$\checkmark$	$\times$	$\times$ (overflow)
value shift, post-norm	$\checkmark$	$\times$	$\times$	$\times$
$K = N$ (linear carry), post-norm	$\checkmark$	$\times$	$\times$	$\times$
$K = N$ (linear carry), no norm	$\checkmark$	$\checkmark$	$\times$	$\checkmark$
value shift, no norm	$\checkmark$	$\checkmark$	$\times$	$\checkmark$ (slow)
code residual	$\checkmark$	$\checkmark$	$\checkmark$	$\checkmark$

The third column is idempotence: TopK is the Euclidean projection onto the K -sparse set, $\text{TopK}(c) = c$ for K -sparse c , so carrying the code makes a silent block the identity. Re-encoding the decoded stream does not, because $W_E W_D \neq I$ (and cannot be on 512-sparse codes in $d = 1024$ with $N = 4096$: the leakage of $W_E^\top W_D$ is comparable to the code at $K/d = 1/2$); every block is a new random rank- K map, which preserves energy on average but spreads the singular values of the product over depth. The value shift and the linear carry satisfy the first two requirements and train; the code residual satisfies all three and trains fastest (Entry 8).

The non-idempotent carry, measured (analysis/carry_spectrum.py: exact Jacobian $\partial x_L / \partial x_0$ of the carry alone, branches off, fresh Gaussian bottlenecks at the MD init statistics, 3 draws). Effective rank $(\sum \sigma^2)^2 / \sum \sigma^4$ out of $d = 1024$ and mean σ^2 :

depth L	1	4	8	16	24
value shift, no norm: effective rank	256	78	41	21	14
mean σ^2	1.00	0.99	0.99	1.00	1.02
TopK + post-norm: mean σ^2	0.26	$3.9 \cdot 10^{-3}$	$1.5 \cdot 10^{-5}$	$2.1 \cdot 10^{-10}$	$2.9 \cdot 10^{-15}$
TopK, no norm: effective rank	255	8.1	1.01	1	1
code residual (silent blocks)	identity: all $\sigma = 1$, effective rank 1024				

Measured: the balanced carry keeps the mean σ^2 at 1, but its energy concentrates on ever fewer directions (effective rank $\approx 256 \cdot L^{-0.9}$); after 24 blocks the stack transmits the input along about 14 of 1024 directions. The post-norm carry has the same spectrum shape scaled by s^{-2L} ; the plain carry is a single spike along the signal (the selection gain). **Interpretation:** this is the information the value-shift model has to re-inject through attention and the MLPs at every block, and the likely reason it trains more slowly than the code residual, whose carry is the identity.

9 Entry 8 — round 2: controls, depth, and the value shift in training

Sparsity-free controls ($K = N$, 24 layers, 2000 steps): with the post-norm val CE 5.959 (plateau, collapsed at step 2000: cosine 1.000, gradient $\times 2.64$ per block); without it 2.072. The post-norm alone decides the outcome for the same carry.

Depth sweep (val CE at step 2000):

layers	8	12	16	24
TopK, post-norm	1.862	5.958	5.959	5.959
TopK, no norm	2.340	3.210	10.825 ($\ln V$, guard stop 1240)	10.825 ($\ln V$)
code residual	1.890	1.860	1.839	1.814

Measured: with the post-norm the stack trains at 8 layers — better than without a norm — and sits on the plateau from 12 layers on, although at 12 layers the initial gradient ratio between the last and the first block is only $\sim 2 \cdot 10^3$. Without a norm the plain stack slows at 12 layers and overflows at 16. The code residual improves monotonically with depth. **Interpretation:** the collapse, not the size of the initial gradient ratio, sets the post-norm threshold: at 8 blocks the lower blocks learn token features before the top blocks’ unigram output locks the collapse in; from 12 blocks on they do not.

The value shift drifts in training (li_24L_k512_nopnorm_shift_energy). Val CE 2.859 (2k), 2.426 (3.5k), 2.137 (6k), with spikes at single evaluations (2.527 at 5k, 2.730 at 6.5k) and raw gradient norms up to 15–23 (clipped at 1). The fraction of kept features that the shift clamps to zero rises steadily: 0.15 (1k), 0.26 (2k), 0.43 (3k), 0.51 (5k), 0.56 (6.9k), with δ/σ from 1.04 to 1.95: the support is effectively ~ 225 of $K = 512$. The **fixed** rule ($\delta = \lambda^* \sigma$ throughout) stalls instead (val 3.764, 3.575, 3.619 at 1k, 1.5k, 2k), and a shared dictionary does not help (3.443 at 1.5k against 3.205). **Interpretation:** the shift removes the selection gain of *Gaussian* codes. Training makes the kept magnitudes heavy-tailed, the selection gain of a trained code grows toward $N/K = 8$, and the energy-matching shift must remove ever more of it (clamping), while a fixed shift falls out of balance. The value shift therefore confirms the mechanism (it trains at 24 layers where plain TopK cannot) but is not a robust fix; the code residual, which never reselects from a fresh encoding, has none of these problems.

The 20k runs (val CE at step 20 000): code residual 1.2658 (seed 2: 1.2668); code residual with an encoder/decoder pair per block (**share_projections=false**, 563M parameters, 1.06s/step) 1.2628, 1.797 at 2k; value shift without a norm 1.5063; code residual at $K = 32$ 1.517. **Measured:** per-block projections are ahead of the shared pair by 0.002–0.003 from step 17k on, three times the seed difference, for 55% more parameters.

10 Note on how the two fixes were arrived at

Both fixes were designed against the first mechanism (Entry 3), before the second was known. The value shift equalizes G_f and G_b directly. The code residual came from the residual principle: a

deep stack trains when every block is the identity plus a small update, and the stream bottleneck breaks that because it re-encodes the stream and reselects from fresh Gaussian coefficients in every block — which is where the selection gain comes from. Carrying the code makes TopK idempotent on its input, so a silent block is the identity and there is no fresh reselection. It was run without a post-norm from the start, as a pre-norm residual stream would be. The second mechanism was found afterwards, when the value shift failed with the post-norm and trained without it (Entry 4). In hindsight the recursion (7) states what the second mechanism requires — a carry that is not renormalized, so that r_ℓ falls with depth — and both no-norm fixes meet it for the same reason: neither normalizes the carry. What the code residual adds is the identity carry (Entry 7).

11 Implementation notes

- `value_shift` works on the K gathered values (only σ^2 needs all N); gate forward+backward at $8 \times 512 \times 4096$ bf16: 2.2ms against 1.0ms for plain TopK (7.9ms for the first, dense implementation, which the round-1 shift runs used).
- A latent bug found on the way: g_D is a plain attribute that only `md_init_` set, so `load_for_inference` and the analysis `--ckpt` paths evaluated every bp checkpoint with $g_D = 1$ — harmless behind a post-norm, a different network without one (e.g. the 8-layer reference). The controller now restores it at install (`f9a869c`); training is unchanged (`tools/repro_check.py` identical in all three modes).